Projektarbeit

Fachbereich M – Master Maschienenbau Mechatraonics | WS 2025/2026

Entwicklung, Integration und Validierung einer ROS2-basierten Hinderniserkennung mit Reaktionsstrategie für automatisierte Modellfahrzeuge

Autoren: Aly Elbermawy (5335848)
Omar Elalami (5379642)
Zakaria El-aboudi (5223183)

Betreuer: Prof. Dr. Benedikt Lattke

Abgabedatum: 9. April 2026

Kurzfassung

Diese Projektarbeit beschreibt die Integration, Dokumentation und Bewertung eines scan-basierten *Follow-the-Gap*-Stacks (FTG) für das MXCarkit. Das Fahrzeug basiert auf einem NVIDIA-Jetson-System, arbeitet mit einem 2D-LiDAR und wird über eine Ackermann-Steuerkette mit VESC angesteuert. Ziel der Arbeit war es, einen robusten, nachvollziehbaren und sicher testbaren ROS 2-Datenpfad von `/scan` bis `/autonomous/ackermann_cmd` aufzubauen.

Die aktualisierte Zielarchitektur besteht aus vier logisch getrennten Stufen: (i) einer TF-basierten Vorverarbeitung des LiDAR-Scans, (ii) dem C++-Planerkern `follow_the_gap_v0` im Scan-Modus, (iii) einem Python-Planer für Lenkwinkel- und Geschwindigkeitsvorgabe und (iv) einem Ackermann-Command-Knoten als Ausgabe in die bestehende MXCK-Steuerkette. Der bisherige Legacy-Pfad über `obstacle_substitution` bleibt kompatibel im Repository, ist jedoch nicht mehr der primäre Laufzeitpfad.

Die theoretische Einordnung des Verfahrens erfolgt anhand der Originalarbeit von Sezer und Gokasan zum Follow-the-Gap-Verfahren, der CTU-Implementierung für F1/10-Fahrzeuge sowie der ROS 2/F1TENTH-Migrationsarbeit von Endler. Auf dieser Basis wird gezeigt, wie sich die klassische FTG-Idee (largest gap, gap center, final heading) in eine aktuelle ROS 2-Pipeline für ein Jetson-basiertes Lehr- und Versuchssystem übertragen lässt.

Schlüsselwörter: Follow-the-Gap, ROS 2, MXCarkit, LiDAR, Jetson, Ackermann, Hindernisvermeidung, autonome Fahrfunktionen

Inhaltsverzeichnis

Kurzfassung **II**

Inhaltsverzeichnis **III**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

1 Grundlagen **1**

1.1 Das MXCarkit als Zielplattform 1

1.2 ROS 2: Workspace, Paket, Node und Launch 2

1.2.1 Workspace 2

1.2.2 Paket 2

1.2.3 Node 2

1.2.4 Launch-Dateien und YAML-Parameter 2

1.3 Follow-the-Gap als reaktiver Planungsansatz 2

1.3.1 Zentrale Idee der Originalarbeit 3

1.3.2 Gap-Idee in vereinfachter Form 3

1.3.3 Von der Originalformulierung zur praktischen Scan-Variante . . 3

1.4 Warum ROS 2 für dieses Thema relevant ist 4

1.5 Zusammenfassung 4

2 Systembeschreibung und Durchführung **5**

2.1 Zielarchitektur des aktuellen Stacks 5

2.2 Paketübersicht 6

2.3 Stufe 1: LiDAR-Vorverarbeitung 6

2.3.1 `scan_preprocessor_node` 6

2.3.2 `scan_front_window_check` 7

2.4 Stufe 2: FTG-Kern im Scan-Modus 7

2.4.1 Ein- und Ausgänge 7

2.4.2 Bedeutung für das Projekt 7

2.5 Stufe 3: Planner 8

2.5.1 Eingänge 8
2.5.2 Ausgänge 8
2.5.3 Aufgabe des Planers 8
2.6 Stufe 4: Control 8
2.6.1 Eingänge und Ausgang 9
2.7 Bringup und Startlogik 9
2.8 Testmethodik 9
2.9 Abgrenzung zu Legacy-Komponenten 10

3 Diskussion und Bewertung 11
3.1 Warum der direkte Scan-Pfad die richtige Hauptarchitektur ist 11
3.2 Rolle von TF und Geometrie 11
3.2.1 Warum `front_center_deg = 0.0` fachlich richtig ist 11
3.2.2 Folgen falscher Frame-Annahmen 12
3.3 Bewertung der Planner-/Control-Aufteilung 12
3.3.1 Vorteile dieser Trennung 12
3.3.2 Mögliche Schwächen 12
3.4 Einordnung der CTU- und ROS 2-Quellen 13
3.4.1 Beitrag der Original-FTG-Publikation 13
3.4.2 Beitrag der CTU-Implementierung 13
3.4.3 Beitrag der ROS 2-/F1TENTH-Thesis 13
3.5 Dokumentationsqualität als Teil der Systemqualität 13
3.6 Grenzen des aktuellen Stands 14

4 Fazit und Ausblick 15
4.1 Fazit 15
4.2 Ausblick 15
Schlussbemerkung 16

Literaturverzeichnis 17

Literaturverzeichnis 17

Abbildungsverzeichnis

Tabellenverzeichnis

1.1 Relevante MXCK-Systemeigenschaften für diese Arbeit 1

2.1 Pakete des aktuellen FTG-Stacks 6

1 Grundlagen

Dieses Kapitel beschreibt die fachlichen und systemtechnischen Grundlagen der Arbeit. Zuerst wird das MXCK als Zielplattform eingeordnet. Danach werden die für die Arbeit relevanten ROS 2-Begriffe erklärt. Abschließend wird der Follow-the-Gap-Algorithmus sowohl in seiner ursprünglichen Form als auch in der für dieses Projekt relevanten scan-basierten Variante erläutert.

1.1 Das MXCarkit als Zielplattform

Das MXCarkit ist ein Jetson-basiertes, containerisiertes Versuchsfahrzeug für Lehre und Entwicklung im Bereich autonomer Fahrfunktionen. Laut der aktuellen MXCK-Dokumentation werden die Systembestandteile bewusst in Docker-Container aufgeteilt, wobei `mxck2_ws` die Kernfunktionalität und `development_ws` experimentelle bzw. GPU-nahe Entwicklung aufnimmt. Die zentrale Startlogik für Sensorik und Aktorik liegt im Paket `mxck_run`; einzelne Teilfunktionen wie LiDAR oder Kamera werden über Launch-Argumente wie `run_lidar:=true` aktiviert.

Tabelle 1.1: Relevante MXCK-Systemeigenschaften für diese Arbeit

Aspekt	**Bedeutung für den FTG-Stack**
NVIDIA Jetson	Rechnerplattform für ROS 2, Paket-Build, Sensorverarbeitung und Debugging
Docker-basierter Betrieb	Trennung von Sensor-/Control-Containern und Entwicklungsumgebung
2D-LiDAR (`/scan`)	Primärer Sensor für den reaktiven FTG-Pfad
Ackermann-Steuerung	Lenkwinkel- und Geschwindigkeitsvorgaben werden als Ackermann-Befehl ausgegeben
Bestehende Fahrzeugsteuerung	Der FTG-Stack darf die vorhandene VESC-/Safety-Kette nicht umgehen

Für diese Arbeit ist vor allem wichtig, dass der autonome Output nicht direkt an VESC-Topics gesendet wird, sondern an `/autonomous/ackermann_cmd`. Dadurch bleibt die bestehende MXCK- Sicherheits- und Moduslogik erhalten.

1.2 ROS 2: Workspace, Paket, Node und Launch

Da der gesamte Stack auf ROS 2 basiert, ist ein sauberes Verständnis der Architekturelemente notwendig.

1.2.1 Workspace

Ein ROS 2-Workspace ist ein Verzeichnis, in dem mehrere Pakete gemeinsam gebaut werden. Gebräuchlich ist die Struktur `src/`, `build/`, `install/`, `log/`. Im MXCK-Projekt wird dieser Aufbau direkt genutzt: `mxck2_ws` enthält die Kernfunktionalität, während `development_ws` für zusätzliche Entwicklungsarbeit vorgesehen ist.

1.2.2 Paket

Ein Paket bündelt eine klar abgegrenzte Funktion. In dieser Arbeit werden Pakete für Perception, Planner, Control und Bringup verwendet. Für Python-Pakete sind insbesondere `package.xml`, `setup.py`, `setup.cfg`, der Python-Modulordner und optionale Launch-/Config-Ordner relevant. Für C++-Pakete kommen stattdessen `CMakeLists.txt`, `package.xml`, Header- und Source-Dateien zum Einsatz.

1.2.3 Node

Ein Node ist ein laufender Prozess innerhalb des ROS 2-Graphs. Nodes abonnieren Topics, veröffentlichen Topics, verwenden Parameter und werden meist über Launch-Dateien gemeinsam gestartet.

1.2.4 Launch-Dateien und YAML-Parameter

ROS 2 nutzt bevorzugt Python-basierte Launch-Dateien. Diese starten mehrere Nodes, reichen Parameter weiter und erlauben deklarative Startkonfigurationen. Für dieses Projekt ist das besonders wichtig, weil die Pipeline nur dann stabil und reproduzierbar ist, wenn Node-Namen, Executables, YAML-Dateien und Topics konsistent zusammenpassen.

1.3 Follow-the-Gap als reaktiver Planungsansatz

Der Ursprung des Follow-the-Gap-Verfahrens liegt in der Arbeit von Sezer und Gokasan. Dort wird FTG als lokaler Hindernisvermeidungsansatz formuliert, der die Umgebung nur anhand der aktuell sichtbaren Hindernisse bewertet. Statt eine globale Karte aufzubauen, sucht das Verfahren in jedem Zeitpunkt nach der größten passierbaren Lücke und richtet das Fahrzeug möglichst auf deren Mitte aus.

1.3.1 Zentrale Idee der Originalarbeit

Die Originalarbeit beschreibt drei Kernschritte:

1) Aufbau einer Hindernis- bzw. Gap-Repräsentation,
2) Bestimmung eines sicheren Gap-Center-Winkels,
3) Fusion von Gap-Richtung und Zielrichtung zu einem finalen Heading.

Wesentliche Eigenschaften des Verfahrens sind laut Originalarbeit:

- Berücksichtigung des Sichtfelds,
- Berücksichtigung nicht-holonomer Fahrdynamik,
- keine lokale-Minimum-Problematik wie bei Potentialfeldern,
- nur wenige Tuning-Parameter.

1.3.2 Gap-Idee in vereinfachter Form

Im klassischen FTG werden aus den Hindernisgrenzen aufeinanderfolgende "Gaps" bestimmt. Die größte geeignete Lücke wird ausgewählt. Aus dieser Lücke wird ein Winkel φ_{gap} bestimmt. Anschließend wird dieser mit einem Zielwinkel φ_{goal} fusioniert, wobei nahe Hindernisse den Einfluss des Gap-Winkels erhöhen und freie Umgebung den Einfluss des Zielwinkels stärkt.

Formal beschreibt die Originalarbeit die finale Heading-Bildung als gewichtete Fusion aus Gap-Winkel und Zielwinkel. Entscheidend ist dabei der kleinste Hindernisabstand d_{min}: je kleiner d_{min} ist, desto stärker soll die Planung sicherheitsorientiert in Richtung des Gaps reagieren.

1.3.3 Von der Originalformulierung zur praktischen Scan-Variante

Die CTU-F1/10-Implementierung übernimmt die FTG-Idee in einen ROS-Node `follow_the_gap`. Dessen klassischer ROS-Eingang ist ein `sensor_msgs/LaserScan` auf `/scan`; als Zielwinkel kann optional `/lsr/angle` vorgegeben werden. Als wesentliche Ausgänge nennt das CTU-Repository `/final_heading_angle`, `/visualize_final_heading_angle`, `/visualize_obs` und `/visualize_largest_gap`.

Für das MXCK-Projekt wird dieser Kerngedanke nicht als `/scan`-Direkteingang verwendet, sondern als *scan-basierter Vorverarbeitungspfad*:

```
/scan
  -> scan_preprocessor_node
  -> /autonomous/ftg/scan_filtered
  -> follow_the_gap_v0 (input_mode=scan)
  -> /final_heading_angle
  -> /gap_found
```

Damit wird der eigentliche C++-Planerkern entlastet: Das Re-Centering auf die Fahrzeugfront, die Clipping-Logik und die Front-Clearance-Bestimmung finden bereits vorher statt.

1.4 Warum ROS 2 für dieses Thema relevant ist

Die CTU-Arbeit von Endler zeigt, warum ROS 2 im Kontext schneller autonomer Fahrfunktionen relevant ist. Zentrale Punkte sind:

- DDS-basierte Kommunikation statt ROS 1-Master-Modell,
- klare Trennung von `rcl`, `rmw` und DDS,
- Build-System `ament` mit `colcon`,
- moderne Python-Launchfiles,
- bessere Grundlage für Messung von Latenzen, Jitter und End-to-End-Ketten.

Für diese Projektarbeit ist das vor allem deshalb wichtig, weil das MXCK nicht nur einen Algorithmus "irgendwie ausführt", sondern als Prozesskette aus mehreren ROS 2-Nodes verstanden werden muss. Schon kleine Inkonsistenzen in Topic-Namen, QoS, Frames oder Startreihenfolge können das Gesamtsystem unbrauchbar machen.

1.5 Zusammenfassung

Die theoretische Grundlage dieser Arbeit ist damit zweigeteilt: Einerseits liefert die FTG-Originalliteratur die mathematische und planungslogische Basis. Andererseits liefert der aktuelle ROS 2-/F1TENTH- Kontext die technische Form, in der diese Idee heute auf einem Jetson-basierten Lehrfahrzeug umgesetzt wird. Genau an dieser Schnittstelle setzt die vorliegende Arbeit an.

2 Systembeschreibung und Durchführung

Dieses Kapitel beschreibt die tatsächlich umgesetzte FTG-Architektur im aktuellen Repository `ftg_mxck`. Der Fokus liegt dabei auf dem scan-basierten Standardpfad, der gegenüber älteren Legacy-Varianten mit `obstacle_substitution` bewusst bevorzugt wird.

2.1 Zielarchitektur des aktuellen Stacks

Die aktuelle Root-Dokumentation des Repositories definiert die Primärpipeline klar als direkten Scan-Pfad bis zur Ackermann-Ausgabe. Daraus ergibt sich folgende Soll-Architektur:

```
/scan (LaserScan, frame: laser)
  -> scan_preprocessor_node                [mxck_ftg_perception]
  -> /autonomous/ftg/scan_filtered         [LaserScan, frame:
     base_link]
  -> /autonomous/ftg/front_clearance       [Float32]

/autonomous/ftg/scan_filtered
  -> follow_the_gap_v0                     [C++, input_mode=scan]
  -> /final_heading_angle                  [Float32]
  -> /gap_found                            [Bool]

/final_heading_angle + /gap_found + /autonomous/ftg/
   front_clearance
  -> ftg_planner_node                      [mxck_ftg_planner]
  -> /autonomous/ftg/gap_angle             [Float32]
  -> /autonomous/ftg/target_speed          [Float32]
  -> /autonomous/ftg/planner_status        [String]

/autonomous/ftg/gap_angle + /autonomous/ftg/target_speed
  -> ftg_command_node                      [mxck_ftg_control]
  -> /autonomous/ackermann_cmd             [AckermannDriveStamped]
```

Wichtig ist dabei, dass die Ackermann-Ausgabe bewusst nicht direkt in niedrigere VESC-Topics schreibt, sondern sich in die bestehende MXCK- Fahrzeugsteuerung einfügt.

2.2 Paketübersicht

Tabelle 2.1: Pakete des aktuellen FTG-Stacks

Paket	Wesentliche Nodes	Rolle im Datenpfad
`mxck_ftg_perception`	`scan_preprocessor_node`, `scan_front_window_check`	TF-basierte Scanvorverarbeitung und Diagnose
`follow_the_gap_v0`	`follow_the_gap`	C++-FTG-Kern im Scan-Modus
`mxck_ftg_planner`	`ftg_planner_node`	Gap-Winkel- und Geschwindigkeitsplanung
`mxck_ftg_control`	`ftg_command_node`	Ackermann-Ausgabe aus Planner-Topics
`mxck_ftg_bringup`	keine Laufzeit-Node	Startlogik für die komplette Pipeline

2.3 Stufe 1: LiDAR-Vorverarbeitung

2.3.1 `scan_preprocessor_node`

Der `scan_preprocessor_node` bildet den Eintrittspunkt des Produktivpfades. Seine Aufgabe besteht darin, aus dem rohen `/scan`-Topic einen für den FTG-Kern gut interpretierbaren Front-Scan zu erzeugen.

Dazu führt der Node folgende Schritte aus:

1. Abonnement von `/scan`,
2. TF-Lookup von `base_link <- laser`,
3. Bestimmung der Fahrtrichtung im Basisrahmen,
4. Auswahl des Frontfensters (z. B. 100°),
5. Clipping auf gültige Minimal- und Maximalreichweiten,
6. Publikation des gefilterten Scans auf `/autonomous/ftg/scan_filtered`,
7. Publikation des minimalen Frontabstands auf `/autonomous/ftg/front_clearance`.

Die zentrale Designentscheidung ist dabei: `front_center_deg = 0.0` beschreibt "vorne im `base_link`-Frame". Die reale Montagerotation des LiDAR wird ausschließlich durch TF aufgelöst. Dadurch muss kein fester Sensor-Offset mehr in die Dokumentation oder in die Laufzeitlogik eingebrannt werden.

2.3.2 `scan_front_window_check`

Der Diagnose-Node wird optional zugeschaltet und dient nicht der normalen Fahrt, sondern der Kalibrierung und Verifikation. Er hilft dabei, das konfigurierte Frontfenster mit der tatsächlichen Fahrzeugfront abzugleichen und die Wirkung des TF-Setups sichtbar zu machen.

2.4 Stufe 2: FTG-Kern im Scan-Modus

Die aktuelle FTG-Nutzung im MXCK geht bewusst *nicht* über den alten Obstacle-Substitution-Pfad, sondern direkt über einen vorbereiteten `LaserScan`. Der C++-Node `follow_the_gap` wird daher mit `input_mode=scan` gestartet.

2.4.1 Ein- und Ausgänge

Der aktuelle CTU-Kern nutzt konzeptionell:

- einen Scan-Eingang,
- optional einen Zielwinkel,
- einen finalen Heading-Winkel als Ausgabe,
- sowie Visualisierungstopics zur Diagnose.

Im aktuellen MXCK-Stack werden insbesondere folgende Topics relevant:

- Input: `/autonomous/ftg/scan_filtered`
- Output: `/final_heading_angle`
- Output: `/gap_found`
- Debug: `/visualize_obstacles`
- Debug: `/visualize_largest_gap`
- Debug: `/visualize_final_heading_angle`

2.4.2 Bedeutung für das Projekt

Diese Stufe ist fachlich der Kern des Systems: Hier wird aus der lokalen Hindernissituation ein Richtungsvektor berechnet. Die Entscheidung, den Node nicht umzuschreiben, sondern als C++-Kern beizubehalten und nur sauber scan-basiert einzubinden, erhält die algorithmische Nähe zur CTU-/F1TENTH-Vorlage.

2.5 Stufe 3: Planner

Der `ftg_planner_node` fasst die algorithmische Ausgabe des FTG-Kerns mit der Frontabstands-Information aus der Vorverarbeitung zusammen. Er bildet damit die Brücke zwischen lokaler Richtungsplanung und einer fahrbaren, sicheren Sollvorgabe.

2.5.1 Eingänge

- `/final_heading_angle`
- `/gap_found`
- `/autonomous/ftg/front_clearance`

2.5.2 Ausgänge

- `/autonomous/ftg/gap_angle`
- `/autonomous/ftg/target_speed`
- `/autonomous/ftg/planner_status`

2.5.3 Aufgabe des Planers

Der Planer übernimmt drei praktische Aufgaben:

1. Freshness-Prüfung der Eingänge,
2. Begrenzung des Lenkwinkels,
3. Geschwindigkeitsvorgabe abhängig von Clearance und Steering.

Damit ist der Planner keine zweite FTG-Implementierung, sondern eine *Policy-Schicht*: Er interpretiert die Heading-Ausgabe im Kontext von Sicherheitsabstand und Fahrbarkeit.

2.6 Stufe 4: Control

Der `ftg_command_node` transformiert die Planner-Ausgänge in einen Ackermann-Befehl.

2.6.1 Eingänge und Ausgang

- Input: `/autonomous/ftg/gap_angle`
- Input: `/autonomous/ftg/target_speed`
- Output: `/autonomous/ackermann_cmd`

Zusätzlich veröffentlicht der Node ein Status-Topic `/autonomous/ftg/control_status`, das für Debugging und End-to-End- Analyse nützlich ist.

2.7 Bringup und Startlogik

Die empfohlene Startdatei ist `ftg_full_system.launch.py`. Laut aktuellem Repository startet sie den scan-basierten Pfad und lädt dabei die YAML-Konfigurationen der Perception-, Planner- und Control-Stufe. Zusätzlich existiert eine vereinfachte Variante `ftg_scan_path.launch.py`.

Wichtig ist: TF und `vehicle_control` gehören *nicht* automatisch zum FTG-Bringup, sondern werden separat über das MXCK-System gestartet.

2.8 Testmethodik

Für ein robustes Vorgehen wurde der Stack stufenweise verifiziert:

1. TF und LiDAR separat prüfen,
2. nur `scan_preprocessor_node` starten,
3. `follow_the_gap_v0` isoliert testen,
4. Planner prüfen,
5. Control prüfen,
6. Full-Stack testen,
7. erst danach Fahrzeug im Autonomous-Modus fahren.

Diese Reihenfolge ist nicht nur praktisch, sondern auch architekturell sinnvoll: Jeder Fehler kann dadurch einer klaren Stufe zugeordnet werden.

2.9 Abgrenzung zu Legacy-Komponenten

Das Repository enthält weiterhin `obstacle_msgs` und `obstacle_substitution`. Diese Pakete bleiben aus Gründen der Kompatibilität erhalten, gehören aber nicht mehr zum primären Datenpfad. Für die Projektdokumentation bedeutet das:

- Sie dürfen als Legacy- oder Alternativpfad erwähnt werden,
- sie dürfen aber nicht mehr als aktuelle Hauptpipeline dargestellt werden.

Gerade diese Trennung war für die Überarbeitung der Dokumentation zentral, weil ältere Kapitelentwürfe den Legacy-Pfad teils noch als Hauptarchitektur beschrieben.

3 Diskussion und Bewertung

In diesem Kapitel werden die wesentlichen technischen Entscheidungen des aktuellen Stacks diskutiert. Ziel ist nicht nur eine Fehlerliste, sondern eine strukturierte Einordnung der Architektur in Bezug auf Robustheit, Wartbarkeit und Übertragbarkeit auf das MXCK.

3.1 Warum der direkte Scan-Pfad die richtige Hauptarchitektur ist

Eine der wichtigsten Klärungen im Projekt war die Abkehr vom alten Obstacle-Pfad als Standardlaufzeitpfad. Fachlich ist diese Entscheidung aus mehreren Gründen sinnvoll:

1. Der FTG-Kern war ursprünglich scan-basiert formuliert.
2. Die Hindernis-Substitution bringt eine zusätzliche Interpretations- und Parametrierungsschicht ins System.
3. Für "schnell robuster FTG auf dem MXCK" ist ein klarer und kurzer Datenpfad vorteilhaft.

Der direkte Pfad `/scan -> scan_filtered -> follow_the_gap` reduziert Fehlerquellen, verkürzt Debugging-Wege und bleibt näher an den ursprünglichen FTG-Annahmen der CTU/F1TENTH-Implementierung.

3.2 Rolle von TF und Geometrie

Die überarbeitete Dokumentation und die aktuelle Codeausrichtung machen deutlich, dass das Hauptproblem früherer Inkonsistenzen nicht im FTG-Kern selbst lag, sondern in der geometrischen Einordnung der Scandaten.

3.2.1 Warum `front_center_deg = 0.0` fachlich richtig ist

Sobald der Preprocessor mit TF arbeitet, ist die Fahrzeugfront eine Eigenschaft des `base_link`-Rahmens und nicht mehr des rohen LiDAR-Winkelsystems. Der Parameter `front_center_deg = 0.0` ist deshalb eine semantische Aussage über den Fahrzeugrahmen: "vorne ist 0° im Basisrahmen".

Die tatsächliche LiDAR-Montage wird dann ausschließlich durch `base_link <- laser` beschrieben. Das ist robuster, weil Montagefehler oder andere Sensorrahmen nicht in mehreren Paketen und Dokumenten manuell nachgeführt werden müssen.

3.2.2 Folgen falscher Frame-Annahmen

Wenn Dokumentation, Launch-Logik und tatsächliche TF-Struktur nicht zusammenpassen, entstehen typische Symptome:

- falsche Scan-Rezentrierung,
- unplausibler `/final_heading_angle`,
- scheinbar "richtige", aber physikalisch falsch orientierte Marker,
- falsche Interpretation des Frontfensters in Foxglove.

Gerade deshalb muss die Dokumentation dem Code folgen und nicht umgekehrt.

3.3 Bewertung der Planner-/Control-Aufteilung

Die aktuelle Aufteilung in `follow_the_gap_v0` (Heading), `ftg_planner_node` (Policy) und `ftg_command_node` (Ackermann-Ausgabe) ist architektonisch sinnvoll.

3.3.1 Vorteile dieser Trennung

- **C++-Kern bleibt schlank:** Der eigentliche FTG-Kern muss keine Geschwindigkeitsregeln oder MXCK-Hardwaregrenzen kennen.
- **Planner als Policy-Schicht:** Geschwindigkeitsvorgaben können geändert werden, ohne den FTG-Algorithmus selbst anzufassen.
- **Control als Safety-Grenze:** Ackermann-Sättigung und Stop bei stale Inputs liegen in einer klaren, letzten Softwarestufe.

3.3.2 Mögliche Schwächen

Die Aufteilung erzeugt mehr ROS 2-Knoten und damit mehr Kommunikationsstellen. Für ein kleines Versuchssystem ist das tolerierbar, erfordert aber:

- konsistente Topic-Namen,
- korrekte YAML-Einbindung,
- klare Freshness-Regeln,

- reproduzierbare Bringup-Dateien.

Die Dokumentation muss diese Prozesskette präzise abbilden, da schon eine kleine Abweichung zwischen README, Launchfile und Kapiteltext zu realen Fehlstarts führen kann.

3.4 Einordnung der CTU- und ROS 2-Quellen

3.4.1 Beitrag der Original-FTG-Publikation

Die Arbeit von Sezer und Gokasan liefert die fachliche Begründung dafür, warum FTG in engem, unstrukturiertem Umfeld gut funktioniert: Largest-gap-Suche, sichere Heading-Wahl und gewichtete Fusion mit Zielrichtung. Für das MXCK ist besonders wichtig, dass das Verfahren von Natur aus lokal und reaktiv ist und keine vollständige Umgebungsmap benötigt.

3.4.2 Beitrag der CTU-Implementierung

Das CTU-Repository zeigt, wie diese Idee praktisch als ROS-Node umgesetzt wird: Scan-Eingang, Zielwinkel-Option und mehrere Diagnose-Ausgänge. Dadurch wird klar, welche Teile algorithmischer Kern sind und welche Teile in einem fahrzeugspezifischen Stack ausgelagert werden dürfen.

3.4.3 Beitrag der ROS 2-/F1TENTH-Thesis

Die CTU-Arbeit von Endler liefert keine MXCK-spezifische Architektur, aber zwei wichtige Perspektiven:

- warum ROS 2 für autonome Fahrketten interessant ist,
- warum Messung, Tracing und Prozesskettenverständnis entscheidend sind.

Für diese Projektarbeit ist dieser Blick besonders nützlich, weil das MXCK nicht nur algorithmisch, sondern auch als Kommunikationskette betrachtet wird.

3.5 Dokumentationsqualität als Teil der Systemqualität

Ein zentrales Ergebnis der Überarbeitung ist, dass Dokumentation in diesem Projekt nicht bloß “Begleittext” ist, sondern Teil der technischen Qualität. Mehrere frühere Problemstellen waren rein dokumentationsbedingt:

- veraltete Diagramme mit `obstacle_substitution` als Hauptpfad,
- falsche Startreihenfolgen,
- nicht mehr aktuelle Node-Listen,
- uneinheitliche Plattformangaben,
- Workflow-Dateien am falschen Ort,
- versehentlich versionierte LaTeX-Build-Artefakte.

Gerade bei einem ROS 2-Stack mit mehreren Containern, mehreren Workspaces und mehreren Paketen ist eine fehlerhafte Doku praktisch gleichbedeutend mit einer fehlerhaften Betriebsanleitung.

3.6 Grenzen des aktuellen Stands

Trotz der klaren Architektur und der verbesserten Dokumentation bleiben noch offene technische Grenzen:

- Der FTG-Ansatz bleibt ein lokaler, reaktiver Planer.
- Engstellen, Sackgassen und komplexe globale Fahrentscheidungen sind nicht seine Stärke.
- Die jetzige Arbeit dokumentiert den robusten Grundpfad, ersetzt aber keine globale Navigation oder Lokalisierung.
- Für höhere Fahrgeschwindigkeiten werden Laufzeitmessung, Jitter-Analyse und reproduzierbare rosbag-basierte Evaluation wichtiger.

Diese Grenzen sind jedoch keine Schwäche des Projekts, sondern eine saubere Einordnung seines Ziels: ein transparenter, robuster, scan-basierter Hindernisvermeidungsstack für das MXCK.

4 Fazit und Ausblick

4.1 Fazit

Im Rahmen dieser Projektarbeit wurde die vorhandene FTG-Dokumentation an den aktuellen technischen Stand des Repositories `ftg_mxck` und an die MXCK-Umgebung aus `mxck2_ws` angepasst. Der wesentliche inhaltliche Gewinn besteht darin, dass nun nicht mehr mehrere widersprüchliche Architekturbilder nebeneinanderstehen, sondern eine klar definierte Standardpipeline beschrieben wird.

Die überarbeitete Kernarchitektur lautet:

`/scan` → `scan_preprocessor_node` → `/autonomous/ftg/scan_filtered` → `follow_the_gap_v0` → `ftg_planner_node` → `ftg_command_node` → `/autonomous/ackermann_cmd`

Damit wurden folgende Ziele erreicht:

- Die Dokumentation entspricht der aktuellen scan-basierten Hauptpipeline des Repositories.
- Die Rollen von Perception, FTG-Kern, Planner und Control sind sauber getrennt.
- Die theoretische Erläuterung des FTG-Verfahrens wurde auf die Originalliteratur und die CTU/F1TENTH-Umsetzung zurückgeführt.
- Die ROS 2-spezifische Einordnung wurde durch die CTU-Arbeit zur ROS 2-Migration und Prozessanalyse gestärkt.
- Veraltete oder missverständliche Beschreibungen des Legacy-Pfads wurden aus dem Zentrum der Dokumentation entfernt.

Insgesamt ist die Arbeit damit nicht nur eine Beschreibung eines Algorithmus, sondern eine technische Bereinigung und Präzisierung eines gesamten ROS 2-Stacks für ein reales Lehr- und Entwicklungsfahrzeug.

4.2 Ausblick

Auf Basis des nun konsistent dokumentierten Stacks bieten sich die folgenden nächsten Schritte an:

1. **PDF-Build automatisieren:** Die Projektdokumentation sollte über einen korrekt im Repository-Root platzierten GitHub-Action-Workflow automatisch gebaut werden. Gleichzeitig sollten LaTeX-Hilfsdateien konsequent ignoriert werden.

2. **Dokumentation und Code synchron halten:** Änderungen an Pipeline, Topic-Namen, YAML-Dateien oder Launchfiles sollten künftig immer gleichzeitig in README und Ausarbeitung nachgeführt werden.

3. **Rosbag-basierte Evaluation:** Für reproduzierbare Tests empfiehlt sich ein standardisierter Satz an Rosbags mit `/scan`, `/tf`, `/final_heading_angle`, `/gap_found`, Planner- und Control-Topics.

4. **Foxglove-Standardlayout:** Ein gemeinsames Debugging-Layout für `/scan`, `/autonomous/ftg` `/visualize_obstacles`, `/visualize_final_heading_angle`, `/autonomous/ackermann_cmd` und die Status-Topics würde künftige Fehlersuche deutlich beschleunigen.

5. **Erweiterung über lokalen FTG hinaus:** Nach erfolgreicher Stabilisierung des lokalen Hindernisvermeidungsstacks kann die Einbindung globaler Routenplanung, Lokalisierung oder zusätzlicher Sensorfusion betrachtet werden.

Schlussbemerkung

Der wichtigste Erkenntnisgewinn dieser Arbeit ist, dass ein robuster autonomer Stack nicht nur aus einem guten Algorithmus besteht. Er benötigt ebenso eine stimmige Prozesskette, korrekte Frame- und Topic-Semantik, nachvollziehbare Startlogik und eine Dokumentation, die exakt dem laufenden System entspricht. Erst das Zusammenspiel dieser Elemente macht den FTG-Stack auf dem MXCK tatsächlich nutzbar.

Literaturverzeichnis

[1] Volkan Sezer, Metin Gokasan: *A novel obstacle avoidance algorithm: "Follow the Gap Method"*, Robotics and Autonomous Systems, Vol. 60, No. 9, 2012, S. 1123–1134.

[2] CTU-IIG: *f1t-ftg: F1/10 Follow-the-gap ROS node*, GitHub-Repository. `https://github.com/CTU-IIG/f1t-ftg`

[3] Martin Endler: *Using ROS 2 for High-Speed Maneuvering in Autonomous Driving*, Bachelor's Thesis, Czech Technical University in Prague, 2022.

[4] Omar Elalami et al.: *ftg_mxck*, GitHub-Repository. `https://github.com/OmarMElalami/ftg_mxck`

[5] William MX: *mxck2_ws*, GitHub-Repository. `https://github.com/william-mx/mxck2_ws`

[6] Open Robotics: *ROS 2 Documentation*. `https://docs.ros.org`